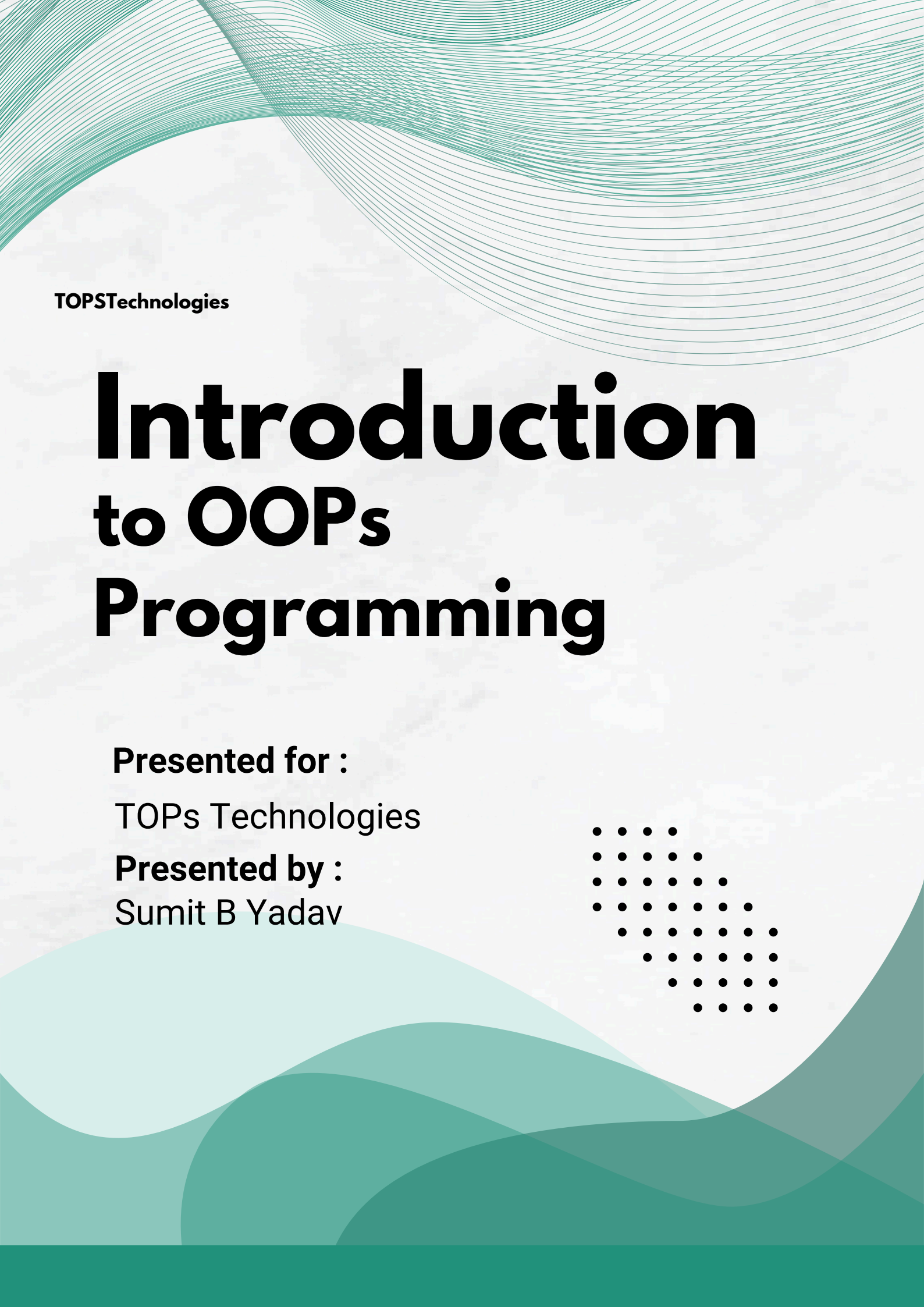




TOPSTechnologies

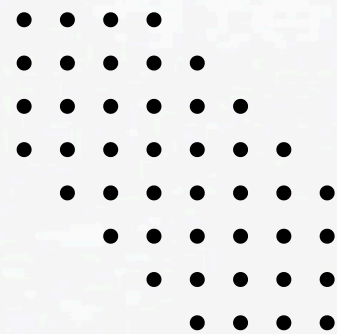
Introduction to OOPs Programming

Presented for :

TOPs Technologies

Presented by :

Sumit B Yadav



Basic Structure of a C++ Program

The key differences between Procedural Programming (POP) and Object-Oriented Programming (OOP) lie in their structure, approach, and design principles. Here's a detailed comparison:

1. Paradigm and Approach

- *Procedural Programming (POP):*
 - *Focus:* Based on the concept of procedures or functions that operate on data.
 - *Approach:* Follows a linear, top-down approach where tasks are broken into functions and executed in sequence.
 - *Data Handling:* Data and functions are separate, and functions manipulate the data passed to them.
 - *Control Flow:* Focuses on controlling the flow of execution using loops, conditionals, and function calls.

- *Object-Oriented Programming (OOP):*
 - *Focus:* Based on the concept of objects which are instances of classes. Objects encapsulate both data (attributes) and methods (functions).
 - *Approach:* Follows a bottom-up approach, where the problem is broken down into smaller, self-contained objects.
 - *Data Handling:* Data and functions are grouped together inside objects. Objects interact with each other through methods.
 - *Control Flow:* Focuses on how objects communicate and interact with each other, often using message passing and methods.

Basic Structure of a C++ Program

2. Structure and Organization

- *Procedural Programming (POP):*

Structure: Code is organized into a series of functions or procedures that operate on data. There's no explicit structure to represent real-world entities.

Modularity: Code can be modular by using functions, but data is often passed around, which can lead to unorganized code as the program grows.

Data Handling: Data is typically global or passed explicitly to functions, which can lead to unstructured access to shared data

- *Object-Oriented Programming (OOP):*

Structure: Code is organized into classes, which define the blueprint for creating objects. Each object can have its own data (attributes) and behaviors (methods).

Modularity: OOP naturally leads to modular code through classes and objects. It encourages reusability and clear separation of concerns.

Data Handling: Data is encapsulated within objects, and access to it is controlled through methods. This helps maintain consistency and prevents unauthorized access.

3. Key Concepts

- *Procedural Programming (POP): Procedures/Functions: Central concept where code is divided into reusable functions. Global Variables: Functions manipulate global or passed variables.*

Control Flow: Primarily uses loops, conditionals, and function calls for managing execution flow.

Basic Structure of a C++ Program

- *Object-Oriented Programming (OOP):*
 - *Classes and Objects: Classes are blueprints for creating objects, which hold both data and behavior.*
 - *Encapsulation: The practice of bundling data (attributes) and methods (functions) that operate on that data inside a class. It also involves controlling access to data through access specifiers (private, public, protected).*
 - *Inheritance: The ability to create new classes from existing ones, inheriting attributes and behaviors.*
 - *Polymorphism: The ability to define methods that can be used in different ways (overloading and overriding).*

Abstraction: Hiding complex implementation details and exposing only necessary functionality through interfaces

4. Data and Function Relationship

- *Procedural Programming (POP):*
 - *Functions and data are separate entities. Functions manipulate data directly, often relying on global variables or parameters passed to them.*
 - *Data is typically stored in variables or arrays, and functions operate on them.*
- *Object-Oriented Programming (OOP):*
 - *Data and functions are encapsulated inside objects. The object's internal state is accessed or modified using methods (functions defined inside the class).*
 - *Objects represent real-world entities, and the class defines the structure and behavior of these entities.*

Basic Structure of a C++ Program

4. Data and Function Relationship Procedural Programming (POP):

Functions and data are separate entities. Functions manipulate data directly, often relying on global variables or parameters passed to them. Data is typically stored in variables or arrays, and functions operate on them. Object-Oriented Programming (OOP): Data and functions are encapsulated inside objects. The object's internal state is accessed or modified using methods (functions defined inside the class). Objects represent real-world entities, and the class defines the structure and behavior of these entities.

- Object-Oriented Programming (OOP):
 - Reusability: High reusability through inheritance (creating new classes based on existing ones) and polymorphism (allowing different behaviors using the same method name).
- Extensibility: OOP is more extensible and maintainable due to its use of inheritance and modular design, allowing easy updates or additions without changing existing code.

6. Maintainability and Modularity

- Procedural Programming (POP):
 - Maintainability: As the program grows larger, procedural code can become difficult to manage due to tightly coupled data and functions.
 - Modularity: While functions can be modular, procedural programs can become harder to understand and modify as the number of functions increases and data is passed around.

Basic Structure of a C++ Program

- *Object-Oriented Programming (OOP):*
 - *Maintainability: Easier to maintain, as the program is divided into well-defined, self-contained objects. Changes to one part of the system are less likely to affect others.*
 - *Modularity: OOP naturally promotes high modularity, as classes are independent units with defined responsibilities.*

Que 2

- *Object-Oriented Programming (OOP) has several advantages over Procedural Programming (POP) in terms of design, maintainability, reusability, and extensibility. Below are the main advantages of OOP over POP:*

1. Encapsulation

- *Explanation: Encapsulation is the concept of bundling data (attributes) and the methods (functions) that operate on the data into a single unit, called a class. The class hides its internal state from the outside world and only exposes a controlled interface to interact with the data, often using getters and setters.*

Basic Structure of a C++ Program

Advantage:

- *Data Protection: Encapsulation protects the internal state of an object from unintended interference or modification. This leads to more secure and robust code.*
- *Modularization: Data and functions are bundled together, which makes it easier to manage and maintain code.*
- *Reduced Complexity: By hiding the internal workings of an object, the programmer only needs to know how to interact with the object, not how it works internally.*

2. Inheritance

- *Explanation: Inheritance allows a class (called a child or subclass) to inherit the properties and methods of another class (called a parent or superclass). The subclass can extend or override the functionality of the parent class.*

Advantage:

- *Code Reusability: Inheritance promotes code reuse by allowing common functionality to be shared between multiple classes. You can create new classes based on existing ones, reducing redundancy and minimizing the amount of new code that needs to be written.*
- *Simplified Maintenance: When common behavior changes, it only needs to be modified in the parent class, and all child classes automatically inherit the updated behavior.*

Basic Structure of a C++ Program

3. Polymorphism

- *Explanation: Polymorphism allows objects of different classes to be treated as objects of a common superclass. The most common forms of polymorphism in OOP are method overloading (same method name but with different parameters) and method overriding (same method name in a subclass, but with different behavior).*

Advantage:

- *Flexibility and Extensibility: Polymorphism allows different objects to respond to the same method call in different ways. This makes the program more flexible and extensible, as new functionality can be added without changing existing code.*
- Code Simplification: A single function or method call can work with different types of objects, simplifying the code and improving maintainability*

4. Abstraction

- *Explanation: Abstraction is the process of hiding complex implementation details and exposing only the essential features of an object or system. In OOP, this is achieved through abstract classes and interfaces, which define the structure and behavior without specifying how the details should be implemented.*

Advantage:

- *Simplifies Interaction: By abstracting away the complexities, users of a class or system can interact with a simpler, high-level interface.*
- *Improved Focus: Developers can focus on what an object does rather than how it does it, leading to cleaner and more understandable code.*
- *Loose Coupling: Abstraction decouples different parts of a system, making it easier to modify or replace components without affecting the rest of the system.*

Que 3

1. Install a C++ Compiler

A C++ compiler is the tool that translates C++ code into executable programs. The most commonly used C++ compilers are:

- **GCC (GNU Compiler Collection):** Popular on Linux and available on Windows and macOS.
- **Clang:** A compiler front end for LLVM, commonly used on macOS and Linux.

Verify Installation:

- Open a Command Prompt (for MinGW) or Developer Command Prompt (for MSVC).
- Type `g++ --version` for MinGW or `cl` for MSVC. If installed correctly, you should see version information for the compiler.

Que 4

Main Input/Output Operations in C++

1. Standard Output (cout)

- `cout` (character output) is used to display information to the standard output (usually the console or terminal).
- The insertion operator (`<<`) is used to send data to the output stream.

2. Standard Input (cin)

- `cin` (character input) is used to read data from the standard input (usually the keyboard).
- The extraction operator (`>>`) is used to extract data from the input stream.